Instituto Tecnológico Superior de Xalapa

Instituto Tecnológico Superior de Xalapa

Estructura de Datos

Tema: Tarea 2 de la unidad 3 Colas estática, circular y dinámica

Alumno: Mauricio Ahued Cortés

Matrícula: 257O02229

Maestro: Víctor Hugo Vásquez Herrera

Xalapa, Veracruz, México
18 de octubre de 2026

Instituto Tecnológico Superior de Xalapa

Índice

Organización del proyecto....2
Cola estática lineal....3
Cola estática circular....4
Cola dinámica....5
Ejecución de la cola dinámica....6
Comparación de las tres colas....7
Resultados de las ejecuciones....8

Las tres colas conservan el orden FIFO: el primero que entra es el primero que sale. La diferencia principal está en cómo guardan los datos y aprovechan el espacio. La lineal puede alcanzar el límite aunque ya se hayan atendido elementos; la circular reutiliza esas posiciones y la dinámica agrega nodos según se necesitan.

Organización del proyecto

Cada programa Main ejecuta una cola distinta. Las clases están organizadas en sus propios paquetes y la cola dinámica utiliza la clase Nodo que ya se trabajó en clase.

```
Estructuras_lineales
  Nodo.java
  COLA_ESTATICA: ColaEstatica y MainColaEstatica
  COLA_CIRCULAR: ColaEstaticaCircular y MainColaCircular
  COLA: ColaDinamica y MainColaDinamica
```

En las colas de arreglo se conservaron los algoritmos de clase y se agregó peek(). La cola dinámica incorpora peek() y contarElementos(). Los campos dato y siguiente de Nodo son públicos para poder usarlos desde el paquete COLA.

Las capturas muestran la ejecución de los tres programas en Visual Studio Code y permiten comparar cómo se insertan, consultan y extraen los datos.

Xalapa, Veracruz, México
18 de octubre de 2026

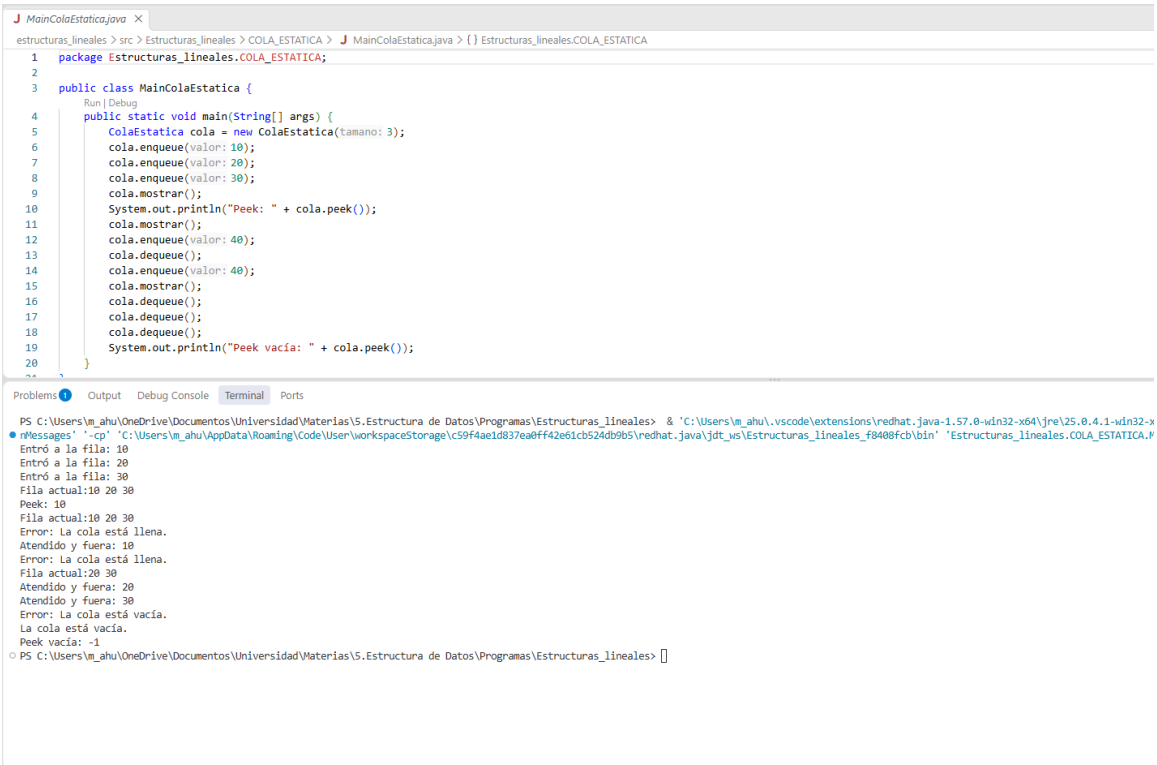
Instituto Tecnológico Superior de Xalapa

Cola estática lineal

El arreglo tiene capacidad tres. frente avanza al atender un dato y fin avanza al insertar. Esta versión no mueve los datos ni recupera las posiciones anteriores: al llegar fin al último índice, las nuevas inserciones se rechazan.

```
public int peek() {
    if (frente > fin) {
        System.out.println("La cola está vacía.");
        return -1;
    }
    return arreglo[frente];
}
```

peek() valida frente > fin y devuelve arreglo[frente] sin cambiar los índices. La fila se imprime igual antes y después de la consulta. Las extracciones atienden 10, 20 y 30 en ese orden.



Después de sacar 10, el segundo intento de insertar 40 también se rechaza. Aunque queda una posición libre al inicio, esta cola lineal no vuelve a utilizarla.

Xalapa, Veracruz, México
18 de octubre de 2026

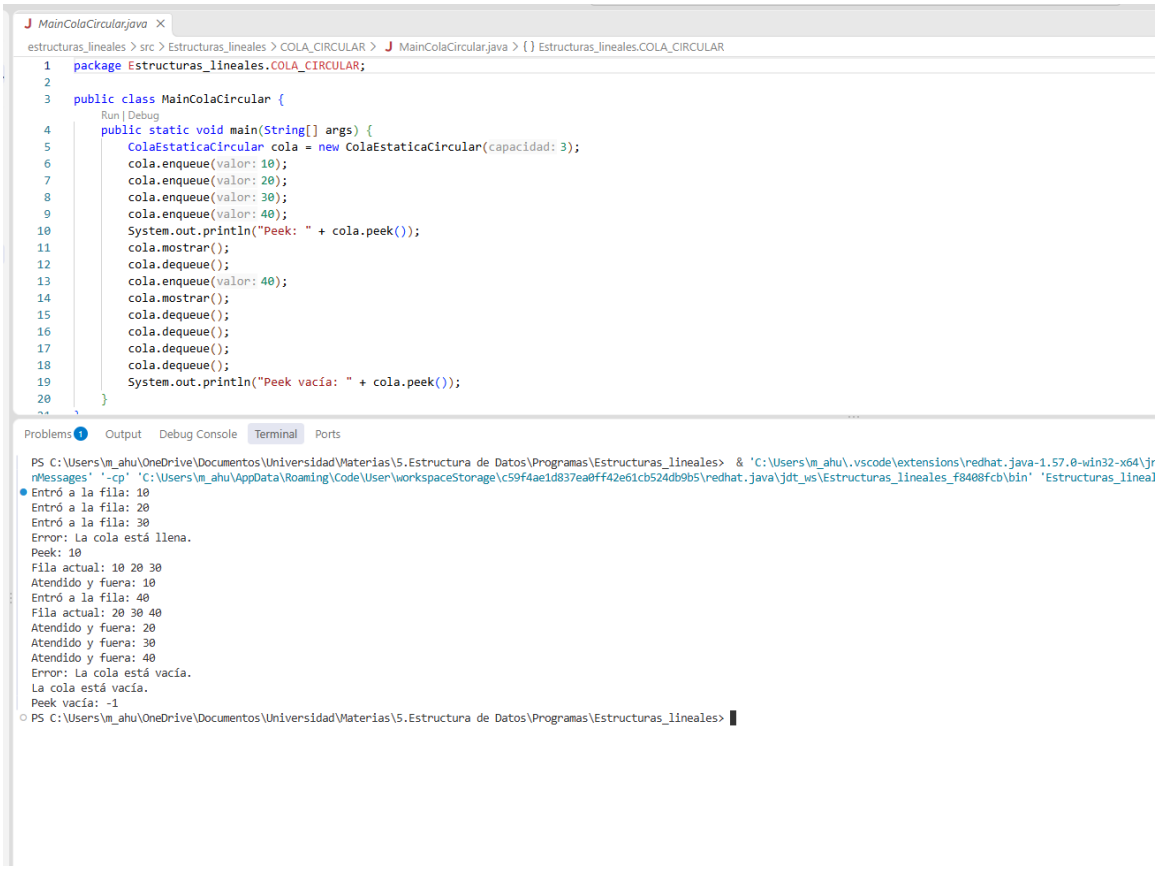
Instituto Tecnológico Superior de Xalapa

Cola estática circular

La capacidad sigue siendo tres. El tamaño indica cuántos datos están activos. Al sacar 10, queda un espacio que se puede usar para 40; la fila lógica pasa a ser 20, 30 y 40.

```
// Avance usado al insertar
fin = (fin + 1) % capacidad;
arreglo[fin] = valor;
tamano++;
// Avance usado al extraer
frente = (frente + 1) % capacidad;
tamano--;
```

```
public int peek() {
    if (tamano == 0) {
        System.out.println("La cola está vacía.");
        return -1;
    }
    return arreglo[frente];
}
```



El módulo regresa el índice al comienzo del arreglo. peek() conserva los índices y el tamaño; la siguiente extracción todavía devuelve 10.

Xalapa, Veracruz, México
18 de octubre de 2026

Instituto Tecnológico Superior de Xalapa

Cola dinámica

Cada dato se guarda en un Nodo. frente apunta al primero y fin al último. Al encolar se enlaza un nodo nuevo al final; al desencolar se avanza frente. Si sale el último nodo, fin vuelve a null para que la siguiente inserción inicie una fila nueva.

```
// Cola con elementos: enlace al insertar
fin.siguiente = nuevo;
fin = nuevo;
// Avance al extraer
int valor = frente.dato;
frente = frente.siguiente;
if (frente == null) {
    fin = null;
}
return valor;

public int peek() {
    if (estaVacia()) {
        System.out.println("La cola está vacía");
        return -1;
    }
    return frente.dato;
}

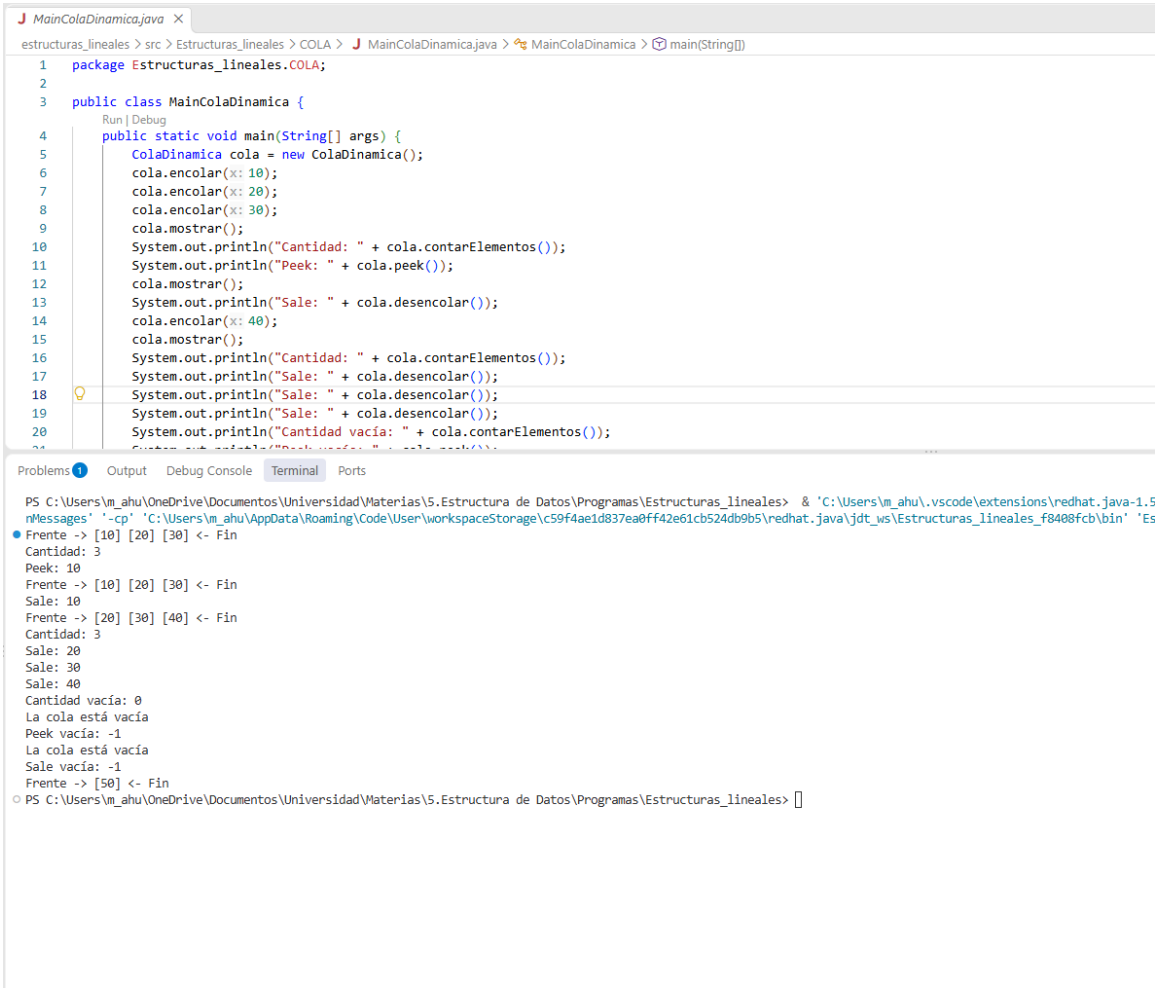
public int contarElementos() {
    int cantidad = 0;
    Nodo actual = frente;
    while (actual != null) {
        cantidad++;
        actual = actual.siguiente;
    }
    return cantidad;
}
```

Para contar los elementos se parte de frente y se sigue el enlace siguiente hasta llegar a null. El conteo se realiza cada vez que se llama al método.

Xalapa, Veracruz, México
18 de octubre de 2026

Instituto Tecnológico Superior de Xalapa

Ejecución de la cola dinámica



```
MainColaDinamica.java
estructuras_lineales > src > Estructuras_lineales > COLA > MainColaDinamica.java > MainColaDinamica > main(String[])
```

```java
package Estructuras_lineales.COLA;

public class MainColaDinamica {
    Run | Debug
    public static void main(String[] args) {
        ColaDinamica cola = new ColaDinamica();
        cola.encolar(x: 10);
        cola.encolar(x: 20);
        cola.encolar(x: 30);
        cola.mostrar();
        System.out.println("Cantidad: " + cola.contarElementos());
        System.out.println("Peek: " + cola.peek());
        cola.mostrar();
        System.out.println("Sale: " + cola.desencolar());
        cola.encolar(x: 40);
        cola.mostrar();
        System.out.println("Cantidad: " + cola.contarElementos());
        System.out.println("Sale: " + cola.desencolar());
        System.out.println("Sale: " + cola.desencolar());
        System.out.println("Sale: " + cola.desencolar());
        System.out.println("Cantidad vacía: " + cola.contarElementos());
```

```
Problems 1  Output  Debug Console  Terminal  Ports

PS C:\Users\m_ahu\OneDrive\Documentos\Universidad\Materias\5.Estructura de Datos\Programas\Estructuras_lineales>  & 'C:\Users\m_ahu\.vscode\extensions\redhat.java-1.5
nMessages' '-cp' 'C:\Users\m_ahu\AppData\Roaming\Code\User\workspaceStorage\c59f4ae1d837ea0ff42e61cb524db9b5\redhat.java\jdt_ws\Estructuras_lineales_f8408fcb\bin' 'Es
Frente -> [10] [20] [30] <- Fin
Cantidad: 3
Peek: 10
Frente -> [10] [20] [30] <- Fin
Sale: 10
Frente -> [20] [30] [40] <- Fin
Cantidad: 3
Sale: 20
Sale: 30
Sale: 40
Cantidad vacía: 0
La cola está vacía
Peek vacía: -1
La cola está vacía
Sale vacía: -1
Frente -> [50] <- Fin
PS C:\Users\m_ahu\OneDrive\Documentos\Universidad\Materias\5.Estructura de Datos\Programas\Estructuras_lineales>
```

La consulta devuelve 10 y la fila conserva sus tres nodos. Después de sacar 10 y agregar 40, el conteo sigue siendo tres. Las extracciones restantes devuelven 20, 30 y 40. Al vaciar la cola, el conteo es cero; luego se inserta 50 correctamente.

Xalapa, Veracruz, México
18 de octubre de 2026

Instituto Tecnológico Superior de Xalapa

Comparación de las tres colas

Las tres permiten consultar el primer elemento sin extraerlo y respetan FIFO. La elección depende de si la cantidad de datos se conoce y de cómo se necesita reutilizar la memoria.

Aspecto	Lineal	Circular	Dinámica
Memoria	Reserva un arreglo completo al crear la cola, aunque tenga pocos datos.	También reserva el arreglo completo. Los índices circulares no amplían el arreglo.	Crea un nodo por inserción. Cada nodo guarda un dato y una referencia siguiente; requiere más estructura por elemento.
Capacidad	Fija. Al llegar fin al último índice, deja de aceptar datos aunque se haya avanzado frente.	Fija. Acepta una nueva inserción siempre que el tamaño sea menor que la capacidad.	No tiene un máximo fijo declarado. El crecimiento depende de la memoria disponible.
Espacio después de extraer	Los espacios anteriores a frente dejan de ser útiles para nuevas inserciones en esta versión.	Los espacios libres se vuelven a usar mediante el avance con módulo.	Se desvincula el nodo atendido. Puede ser recuperado por el recolector de Java cuando ya no tiene referencias.
Operaciones	Insertar, extraer y peek consultan o cambian índices, sin recorrer la fila.	Las mismas operaciones usan índices y tamaño, sin recorrer la fila.	Insertar, extraer y peek usan frente y fin, sin recorrer. contarElementos y mostrar sí recorren los nodos.
Ventaja	Código simple para entender el orden de atención y el manejo del arreglo.	Aprovecha todos los espacios del arreglo en ciclos sucesivos sin desplazar datos.	Se adapta a una cantidad variable de datos y puede volver a crecer después de vaciarse.
Limitación	Pierde espacio utilizable y no reinicia los índices al quedar vacía.	No puede superar la capacidad definida al crearla. Requiere controlar tamaño e índices.	Necesita crear nodos y mantener enlaces. El conteo solicitado tarda más cuando hay más nodos.

Para una fila de tamaño conocido con entradas y salidas continuas, la circular aprovecha mejor el arreglo. La dinámica resulta útil cuando la cantidad cambia y no se quiere fijar una capacidad inicial. La lineal muestra FIFO con una implementación sencilla, pero su límite de espacio la hace menos práctica para un uso continuo.

Xalapa, Veracruz, México
18 de octubre de 2026

Instituto Tecnológico Superior de Xalapa

Resultados de las ejecuciones

En las tres ejecuciones los datos salen en el orden en que entraron. La consulta con peek() muestra el primer dato sin retirarlo. Al vaciar las colas aparecen los mensajes correspondientes; no se interrumpe la ejecución.

Cuando la cola está vacía, peek() muestra un mensaje y devuelve -1. En estas pruebas se usan datos positivos, por lo que ese valor permite reconocer la consulta a una cola sin elementos.

Xalapa, Veracruz, México
18 de octubre de 2026